Running and Reading resnet_cls: Quick Guide

Version 1.0 · 4 November 2026

Introduction

This guide shows how to use `resnet_cls`, a Python package that teaches an existing image-recognition network (ResNet-50 or ResNet-101) to recognise diseases in medical images. You do not need any machine learning background to run it. Section 5 explains the algorithm in plain terms for readers who are new to deep learning.

What goes in. One 2D medical image per prediction, such as a chest X-ray, as a DICOM file or an ordinary PNG or JPG. To train the model you also need examples that are already labelled: the images are sorted into one folder per diagnosis (for example `Normal`, `Pneumonia` and `COVID-19`).

What comes out. For each image, one probability per class, adding up to 1 (for example Normal 0.61, Pneumonia 0.27, COVID-19 0.12), plus the predicted class. Training also produces a saved model (`best.pt`) and a report of how well it works (`metrics.json`), including accuracy, AUC, sensitivity and specificity.

What the algorithm achieves. It starts from a network that has already learned to see from millions of everyday photos and retrains its last layers on your medical images. This approach, called transfer learning, reaches useful accuracy with far less data than training from scratch. It then reports honestly how well the result works on images it has never seen, and rescales the probabilities so they can be taken at face value (calibration). The same code handles other image tasks, such as diabetic retinopathy grading, melanoma detection and histopathology subtypes, by changing the image folders and one setting.

A research and teaching tool. It is not a medical device, and its output is not a diagnosis.

How to read this guide. Sections 1 to 3 cover running the code and reading the files it produces, Section 4 explains how to interpret the numbers, Section 5 explains the algorithm itself, and Section 6 lists its limitations and risks.

Status of this guide: the tests, manifest building and training have been run successfully on the author's Windows machine with a fake dataset. The `predict` and `--resume` commands below had not yet been confirmed when this was written.

1. How to run the whole system

Run every command from the **parent folder** (the folder that contains `resnet_cls/`), never from inside `resnet_cls/`. The layout should look like this:

```
resnet_cls_Dicom/
├── resnet_cls/        the package (config.py, data.py, train.py, ...)
├── raw/               your images, one sub-folder per class
├── venv/              the virtual environment
├── manifest.csv       created in step 7
└── runs/              created by training, one sub-folder per run
```

Step 1. Check Python. `python --version` must show 3.10 or newer.

Step 2. Open a terminal in the parent folder. In File Explorer, open `resnet_cls_Dicom`, click the address bar, type `cmd` and press Enter. Confirm that `resnet_cls/dicom_io.py` exists, which tells you that you have the DICOM version.

Step 3. Create and activate a virtual environment.

```
python -m venv venv
venv\Scripts\activate
```

The prompt should now start with `(venv)`.

Step 4. Install the requirements (a large download, mainly PyTorch):

```
pip install -r resnet_cls/requirements.txt
```

If the three `pylibjpeg` lines fail, delete them from `requirements.txt` and rerun. They are only needed for compressed DICOM files.

Step 5. Run the tests.

```
python -m pytest resnet_cls/tests
```

All tests should pass. The first run takes a minute or two because PyTorch and ResNet-50 are slow to load.

Step 6. Prepare the data. Put images in `raw/<ClassName>/`, for example `raw/Normal/`, `raw/Pneumonia/` and `raw/COVID-19/`. Files can be DICOM (`.dcm`) or ordinary images (PNG, JPG). The patient ID comes from the DICOM `PatientID` field, or for ordinary

images from the part of the file name before the first underscore (`P012_1.png` belongs to patient `P012`). Use at least about 15 different patients per class, or the split will fail.

Step 7. Build the manifest.

```
python -c "from resnet_cls.manifest import build_manifest;
print(build_manifest('raw','manifest.csv',salt='CHANGE_ME'))"
```

It writes `manifest.csv` and prints class counts per split. Every class must appear in train, val and test. The split is by patient, so one patient's images never land in two splits. Keep the same `salt` for the whole project and do not publish it.

Step 8. Train.

Quick smoke test (a few minutes on a CPU):

```
python -m resnet_cls.train --manifest manifest.csv --output-dir runs/smoke --
head-epochs 1 --full-epochs 1 --num-workers 0 --batch-size 4
```

Real run with defaults (3 head epochs, 12 full epochs):

```
python -m resnet_cls.train --manifest manifest.csv --output-dir runs/exp1
```

Useful options: `--arch resnet101` for the deeper model, `--no-grayscale` for colour images (fundus, dermoscopy, histology), and `--dicom-window "-600,1500"` to force a CT window. On a GPU, drop `--num-workers 0` and raise `--batch-size`. Always use a **new** `--output-dir` for each run (see Section 2).

Step 9. Predict on an image or DICOM file.

```
python -m resnet_cls.predict --checkpoint runs/smoke/best.pt
raw/Normal/Nor00_0.png
```

Step 10 (optional). Resume an interrupted run by repeating the training command with the same `--output-dir` and adding `--resume runs/smoke/last.pt`.

Step 11 (optional). Export for deployment:

```
python -m resnet_cls.export --checkpoint runs/exp1/best.pt --out model.onnx
```

Two errors seen in practice:

- `No module named 'resnet_cls'` means you are inside the `resnet_cls` folder. Run `cd ..` and try again.

- `FileNotFoundError ... 'raw'` means the `raw` folder is not in the folder you are running from.

2. How results are stored

Each run writes everything to its `--output-dir`:

File	Format	What it contains	Written
`config.json`	JSON	Every setting used in the run	At the start
`train.log`	Text	The console log, one line per epoch	Each epoch (appended)
`history.jsonl`	JSON Lines	One JSON object per epoch: phase, epoch, training loss, validation metrics, seconds	Each epoch (appended)
`best.pt`	PyTorch checkpoint	The weights of the best epoch plus class names, the full config, the validation AUC and the fitted temperature	When validation AUC improves; temperature added at the end
`last.pt`	PyTorch checkpoint	Everything needed to resume: weights, optimizer, scheduler and scaler state, current phase and epoch, best AUC, early-stopping counter	Every epoch
`metrics.json`	JSON	Final calibrated validation metrics, test metrics (if a test split exists) and the temperature	At the end

Things to know:

- **Text vs binary.** `.json`, `.jsonl` and `.log` open in any text editor. The `.pt` files are binary and can only be read with Python and PyTorch.

- **Which file for what.** Use `best.pt` for prediction and export. `last.pt` is only for resuming and is larger because it also holds optimizer state.
- **Files are appended.** Reusing an output folder adds new lines to `train.log` and `history.jsonl` and mixes two runs together. Use a fresh folder for each experiment (`runs/exp1`, `runs/exp2`, …).
- **A crashed run is incomplete.** If training stops before the final step, there is no `metrics.json` and `best.pt` has no temperature. Do not use it.
- **Only load `.pt` files you trust.** The code loads them with `weights_only=True`, which is the safe mode.

3. How to read the files

Open `config.json`, `metrics.json` and `train.log` in Notepad or any editor. To read everything else, run this in Python from the parent folder:

```
import json, torch

# metrics.json: plain JSON
m = json.load(open('runs/smoke/metrics.json'))
print('temperature:', m['temperature'])
print('val auc:', m['val']['auc'])
print('test auc:', m.get('test', {}).get('auc'))   # None if there was no
test split

# history.jsonl: one JSON object per line, one line per epoch
for line in open('runs/smoke/history.jsonl'):
    h = json.loads(line)
    print(h['phase'], h['epoch'], round(h['train_loss'], 3), round(h['val']
['auc'], 3))

# best.pt: a dictionary
ck = torch.load('runs/smoke/best.pt', map_location='cpu', weights_only=True)
print(ck.keys())                        # model, classes, config, val_auc,
temperature
print(ck['classes'], ck['temperature'], ck['val_auc'])
print(ck['config']['arch'], ck['config']['img_size'])
```

```
# last.pt: training state
st = torch.load('runs/smoke/last.pt', map_location='cpu', weights_only=True)
print(st['phase_idx'], st['epoch'], st['best_auc'])   # phase 0 = head, 1 =
full
```

The structure of `metrics.json` is:

```
metrics.json
├── temperature
├── val
|   ├── acc, auc, nll
|   ├── confusion_matrix
|   └── per_class → <class name> → sensitivity, specificity
└── test        same fields; only present if a test split exists
```

4. How to interpret the final results

Which numbers to trust

1. **`test`** in **`metrics.json`** is the most honest estimate. It was evaluated once, after every decision was final.
2. **`val`** was used to pick the best epoch and to fit the temperature, so it is slightly optimistic.
3. **`history.jsonl`** shows how training evolved. Its validation numbers are measured *before* calibration, so for three or more classes the AUC can differ slightly from `metrics.json`.

What each metric means

Metric	Meaning	Chance level
`acc`	Share of images classified correctly	1 divided by the number of classes, or the majority-class share if classes are imbalanced
`auc`	Macro one-vs-rest AUC: how well the probabilities rank true cases above non-cases, across all thresholds	0.5

Metric	Meaning	Chance level
`nll`	Cross-entropy of the calibrated probabilities. Lower is better	ln(K), which is 1.099 for 3 classes
`sensitivity`	Of the true cases of this class, the share the model found	n/a
`specificity`	Of the images that are not this class, the share correctly left out	n/a
`temperature`	One number that rescales the model's confidence (see below)	1.0 means already calibrated

Reading the confusion matrix

Rows are the true class and columns are the predicted class, in the order of `classes` (alphabetical folder names, for example COVID-19, Normal, Pneumonia). The diagonal is correct answers. Look at the **row sums** too: a class with only a handful of images gives very uncertain sensitivity. Look for a column that collects most predictions, which means the model is defaulting to one class (in the fake-data smoke run it labeled 12 of 14 images "Pneumonia").

Always read sensitivity and specificity together. A class can have high sensitivity and very low specificity simply because the model says that class for everything.

Reading the temperature

The model's logits are divided by this number before the softmax. A value **above 1** means the model was overconfident and its probabilities were softened, which is the usual outcome for a fine-tuned network. Close to 1 means it was already well calibrated. **Below 1** means the model was underconfident. A value far from 1 deserves a look. In the smoke run the value of 0.509 is meaningless, because the model trained on random noise.

Reading the training history

- Training loss falling **and** validation AUC rising: the model is learning.
- Training loss falling but validation AUC flat or dropping: overfitting. Early stopping ends the run after 4 epochs without a gain and logs it.
- Validation AUC jumping around between epochs: the validation set is too small or too noisy to trust.

Reading a prediction

`predict` returns one entry per file, with the file name, the predicted class and a probability for each class:

```
[
  {
    "file": "raw/Normal/Nor00_0.png",
    "prediction": "Normal",
    "probabilities": {"COVID-19": 0.12, "Normal": 0.61, "Pneumonia": 0.27}
  }
]
```

The probabilities are calibrated and add up to 1. A low top probability means the model is unsure. Remember that a softmax model always names one of its classes, even for an image it was never trained on (a lateral view, a different body part, a blank image), so a confident answer is not proof that the input was suitable. Treat the output as a research result, never as a diagnosis.

Before trusting a result

- The split was by patient (the manifest does this).
- Every class has enough test images that its row in the confusion matrix is meaningful.
- You checked sensitivity on the clinically important class, not only the overall accuracy.
- The model was tested on data from a different hospital or scanner than the one it was trained on.

5. How the algorithm works

The algorithm trains in two phases, keeps the best epoch by validation AUC, then calibrates the probabilities and tests once.

A primer on ResNet for Python developers

This note assumes you know Python but nothing about deep learning.

A neural network is a chain of functions. Think of `f3(f2(f1(image)))`, where each function holds a large table of numbers called weights. Training adjusts those numbers. One training step is: run a batch of images through the chain (the forward pass), measure how wrong the answers are with a loss function, work out how much each weight

contributed to the error (backpropagation), then nudge every weight a little in the direction that reduces the error (gradient descent). Repeating this over many batches is what learning means here.

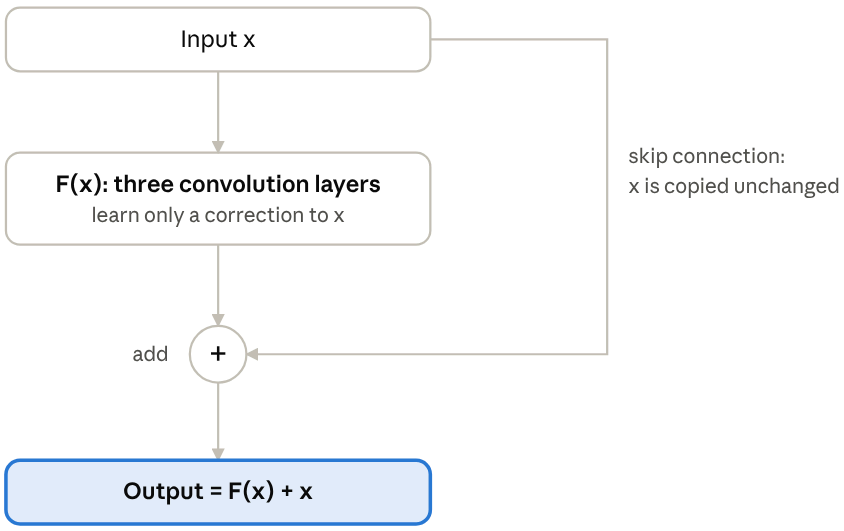
Convolutions look at small patches. Most layers in an image network are convolutions. A small filter, for example a 3 x 3 grid of weights, slides across the image and responds to a pattern. Early layers learn simple patterns such as edges and textures, later layers combine them into shapes, and the last layers respond to whole structures.

The problem ResNet solved. Deeper networks can recognise more, but before 2015 very deep ones became harder to train, and adding layers could even make results worse. ResNet (Residual Network), introduced by researchers at Microsoft Research in 2015, fixed this with one trick, the skip connection. Each building block computes its own output F(x) and then adds its input x back:

```
def residual_block(x):
    out = conv_layers(x)    # F(x): the block's own work
    return relu(out + x)    # add the input back: the skip connection
```

If a block has nothing useful to add, it can learn F(x) close to zero and simply pass x along. The skip also gives the training signal a direct path backward through the network, so networks with 50, 101 or 152 layers train reliably. The diagram shows one block.

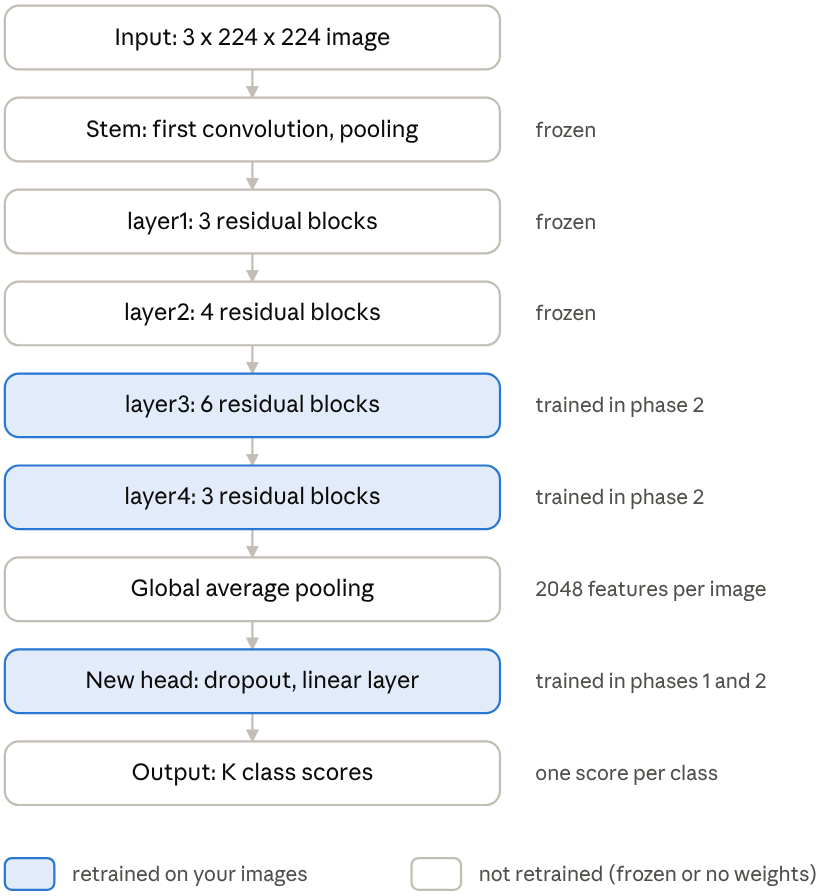
A residual block adds its input back, so its layers only learn a correction



residual block · the skip connection adds the input back

What ResNet-50 means. The 50 counts the layers that have weights: one first convolution, 16 residual blocks of 3 convolutions each, and one final layer (1 + 16 x 3 + 1). The 16 blocks sit in four stages, `layer1` to `layer4`, with 3, 4, 6 and 3 blocks. ResNet-101 follows the same design with more blocks in the third stage (23 instead of 6). After the last stage, global average pooling squeezes each image into a list of 2048 numbers (its features), and a final layer turns those into one score per class.

ResNet-50 is a stack of stages, and we retrain only the last ones



ResNet-50 stages · which parts are retrained

Why we retrain only the end. `resnet_cls` loads a ResNet that someone else already trained on ImageNet, a collection of about 1.2 million everyday photos in 1,000 categories. Its early stages have learned general visual building blocks, and those carry over to X-rays. So we keep them, replace the final layer with a new one sized for our classes (the head), and train only the last parts. This is transfer learning. In phase 1 only the head trains. In phase 2 `layer3` and `layer4` train as well, with small learning rates.

Vocabulary in one table.

Term	Plain meaning	Where you see it
Weights	The numbers inside the network that training adjusts	Saved in `best.pt`
Batch	A small group of images processed together	The `--batch-size` option
Epoch	One full pass over all training images	One line in `train.log`
Loss	One number measuring how wrong a batch of answers is; training tries to lower it	`train_loss` in `history.jsonl`
Backpropagation	Working out how much each weight contributed to the loss	Inside every training step
Frozen layer	A layer whose weights are not updated	Phase 1 freezes the whole backbone
Logits	The raw scores the network outputs, one per class	Just before the softmax
Softmax	Turns scores into probabilities that add up to 1	The probabilities in a prediction
Checkpoint	A saved copy of the weights and training state	`best.pt` and `last.pt`

The training loop in Python

Training is three nested loops: phases, then epochs, then batches. A phase is one stage of the schedule (head only, then deep fine-tuning). An epoch is one pass over every training image. A batch is a small group of images (32 by default) that goes through the network together, and each batch causes exactly one weight update. For example, with 3,200 training images and a batch size of 32, one epoch is 100 batches and therefore 100 weight updates, and the default 3 + 12 epochs make 1,500 updates.

This skeleton keeps the structure and drops the details. The real code is in `resnet_cls/train.py` and `resnet_cls/engine.py`.

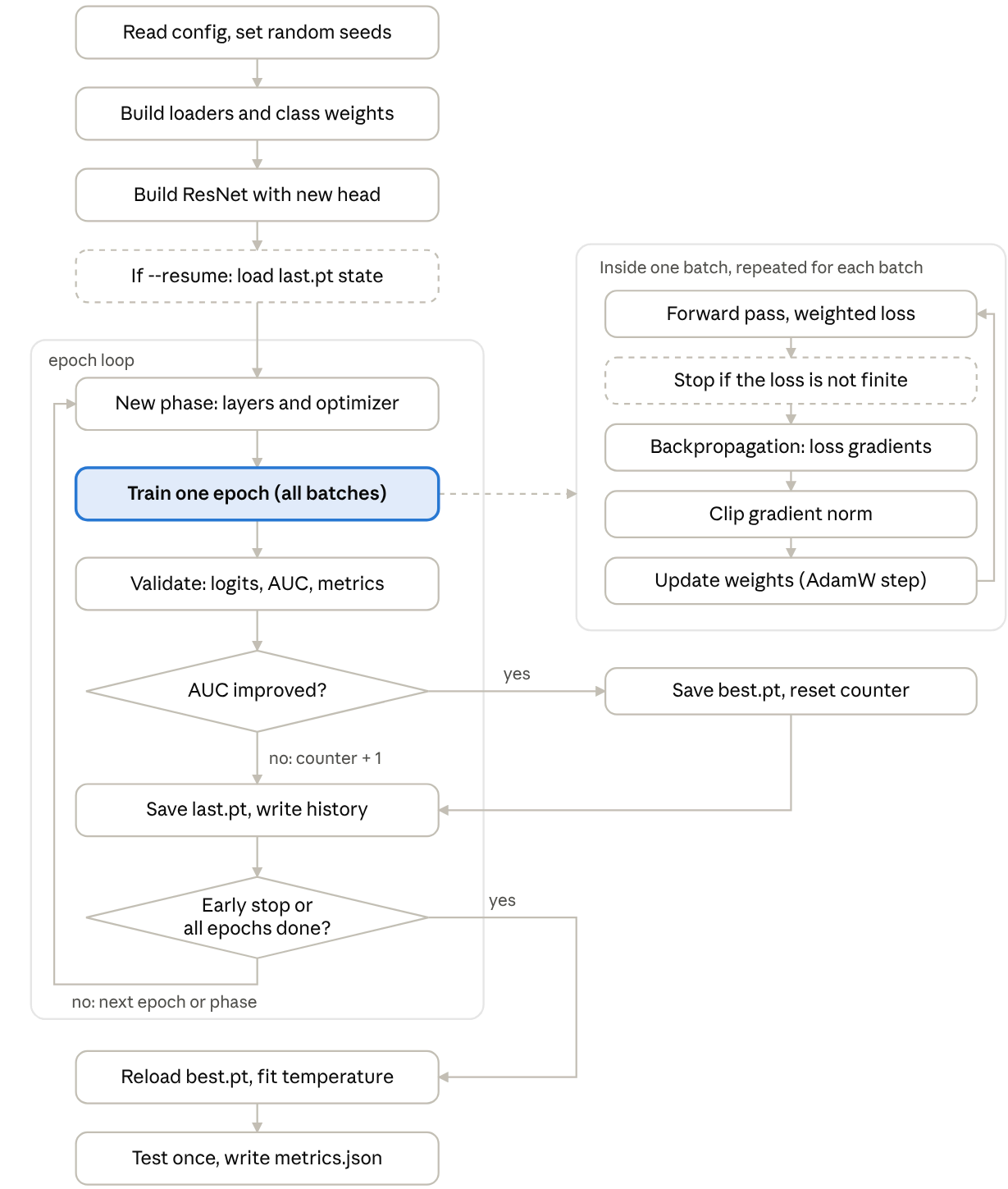
```
for phase in ['head', 'full']:                      # 1. two phases, one
after the other
    n_epochs = head_epochs if phase == 'head' else full_epochs
    set_trainable(model, ['fc'] if phase == 'head' else ['layer3', 'layer4',
'fc'])
    optimizer = build_optimizer(model, phase)       #    freeze or unfreeze,
then build the optimizer

    for epoch in range(n_epochs):                   # 2. one epoch = one
pass over ALL training images
        model.train()                               #    (frozen BatchNorm
layers stay in eval mode)
        for images, labels in train_loader:         # 3. one batch = a group
of images, e.g. 32
            optimizer.zero_grad()
            logits = model(images)                  #    forward pass: all
images in the batch together
            loss = criterion(logits, labels)        #    weighted cross-
entropy: ONE number per batch
            loss.backward()                         #    backpropagation:
gradients of the loss
            clip_grad_norm_(trainable_params, 1.0)  #    clip the gradient
norm
            optimizer.step()                        #    ONE weight update
per batch

        metrics = validate(model, val_loader)       #    no learning here,
only measuring
        if metrics['auc'] > best_auc:               #    keep the best epoch
            best_auc = metrics['auc']
            save('best.pt')
        save('last.pt')                             #    resumable state,
written every epoch
```

The skeleton leaves out mixed-precision handling, the check that stops the run if the loss is not finite, the learning-rate schedule and early stopping (both phase 2 only), and the history file. The flow chart below includes them.

Flow chart of the training algorithm

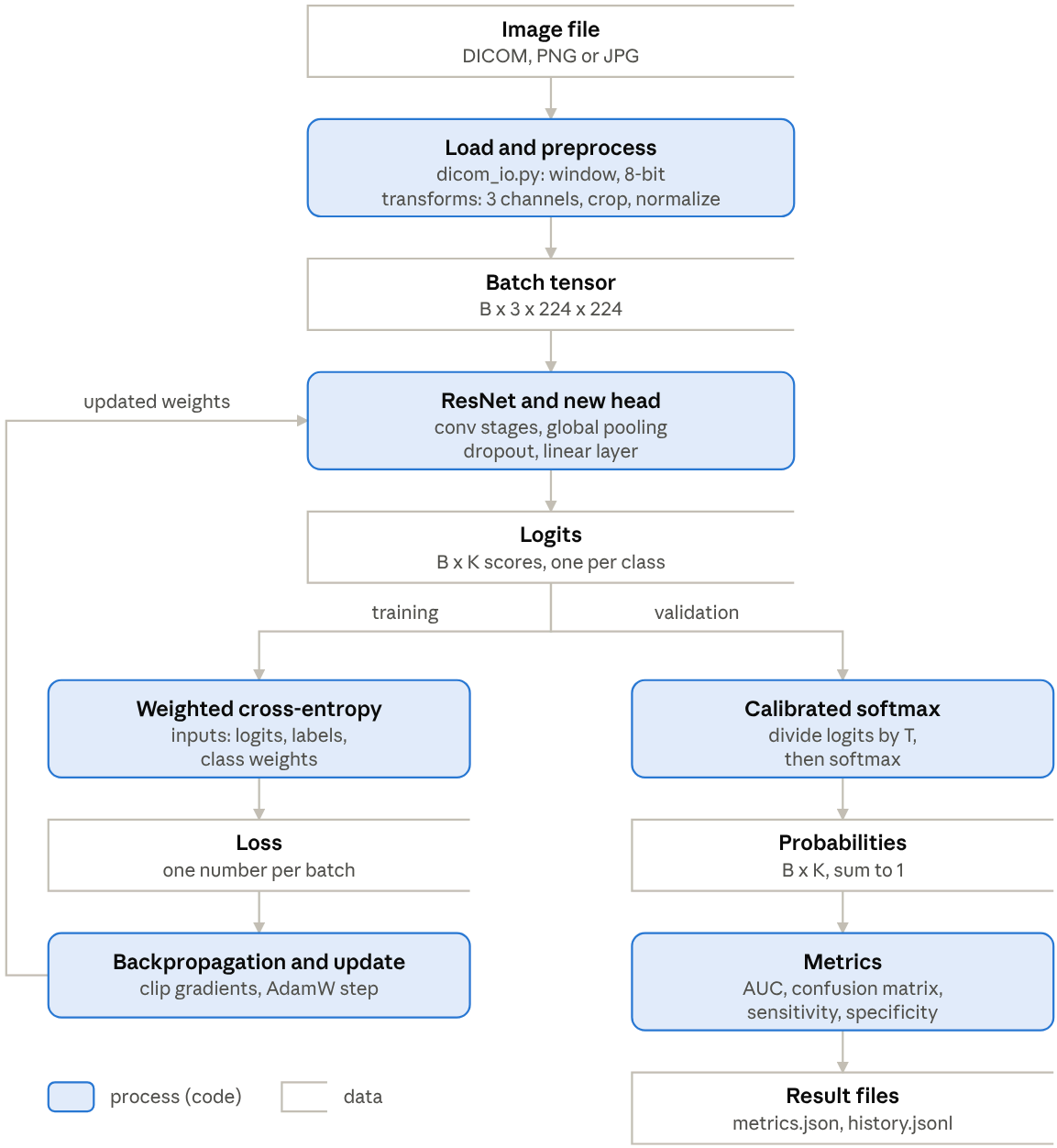


algorithm flow chart · epoch loop with 2 decisions

The outer loop runs over epochs, and the phase changes from head-only to deep fine-tuning inside it. Dashed boxes are optional or protective steps. The panel on the right zooms into one training batch.

Data flow through one batch

Each batch turns images into scores that drive both learning and metrics



data flow · one batch, training and validation branches

B is the batch size and K is the number of classes. The left branch (training) turns the logits into a loss and updates the weights. The right branch (validation) turns the logits into probabilities and metrics. T is 1 during training and is fitted once, after the last epoch.

6. Limitations and risks

This package is a research and teaching tool. Read this section before anyone relies on a model trained with it. Most of the published evidence comes from COVID-19 chest X-ray models, which are the best-documented case, but the same problems apply to other diseases and image types.

The main risk: good scores that do not carry over

The biggest shortcomings come from the data and from how the model is validated, not from ResNet itself. A fine-tuned ResNet can look excellent on your own validation set and still fail in a new hospital.

- **Shortcut learning.** A study of published COVID-19 X-ray models found that they relied on medically irrelevant cues, such as which data source an image came from, instead of disease patterns, and performed worse on an external dataset [2]. In general, a big drop between internal and external testing is a sign that a model is using shortcuts [4].
- **Explanations can expose it.** One review found that the image regions contributing most to a classification often lay outside the lungs, which should carry no diagnostic information [3]. Use the Grad-CAM maps as a check on the model, not as proof that it is right.
- **Flawed datasets.** A systematic review of 62 models found that none was of potential clinical use, because of methodological flaws or biases [1]. In one example the non-COVID images came from children and the COVID images from adults, so a model could succeed just by telling children from adults [1].
- **What the manifest does and does not fix.** Splitting by patient prevents one kind of leakage. It cannot remove confounding in how the data was collected.

Limits of the X-ray itself

- The patterns are subtle and overlap with other viral pneumonias, and radiology societies advise against using this kind of imaging alone for diagnosis [3].
- In mild cases an X-ray may not show the abnormalities at a visible level [5].

- Labels often come from reports or other tests, so some are wrong. That limits what any model can learn.

Limits of this application

Limitation	Why it matters	What helps
One image, one label	No multiple findings at once, and no use of a second view	A multi-label output; combining views per patient
224 px input	Small findings, such as nodules, can disappear when the image is shrunk	Higher resolution or region crops
No none-of-the-above answer	The softmax always names one of your classes, even for an unsuitable image. Only the DICOM modality and view check blocks some bad inputs	An abstain option below a confidence threshold; an out-of-distribution check
No clinical context	Age, symptoms and lab results never reach the model	A separate model that combines them with the image result
Pretraining mismatch	ResNet learned from colour photos. Grayscale is copied to three channels, and the 8-bit conversion discards some intensity range	Test alternatives; a multi-window input for CT
Local calibration	The temperature is fitted on one validation set from one source	Recalibrate on data from each new site
Point estimates only	The metrics have no confidence intervals, so a small test set can swing a lot unnoticed	Report bootstrap confidence intervals
Not run on real data	The only test so far used random-noise images	Train and evaluate on real data before making any claim

Deployment and process

- Validate on data from a different hospital or scanner, report results by subgroup (age, sex, scanner, site), and monitor inputs and predictions after deployment.

- Keep a human in the loop. People tend to trust a confident output (automation bias).
- Remove protected health information from DICOM files before training.
- Clinical use requires regulatory clearance as software as a medical device.

What reduces these risks most

1. Validate on a different hospital or scanner.
2. Report results by subgroup, with confidence intervals.
3. Check the Grad-CAM maps on many images, not a few.
4. Add an uncertain option and an out-of-distribution check.
5. Raise the input resolution if your findings are small.
6. Run a prospective study before any clinical claim.

References

1. Roberts, M. et al. Common pitfalls and recommendations for using machine learning to detect and prognosticate for COVID-19 using chest radiographs and CT scans. Nature Machine Intelligence 3, 199-217 (2021). doi:10.1038/s42256-021-00307-0
2. DeGrave, A. et al. (University of Washington). Study of shortcut learning in COVID-19 chest X-ray models. Nature Machine Intelligence, published 31 May 2021. News release: eurekalert.org/news-releases/729217
3. Lopez Cabrera, J. D. et al. Current limitations to identify COVID-19 using artificial intelligence with chest X-ray imaging (Part II). The shortcut learning problem. Health and Technology 11(2), 411-424 (2021). doi:10.1007/s12553-021-00609-8
4. Ben Ahmed, K. et al. Achieving Multisite Generalization for CNN-Based Disease Diagnosis Models by Mitigating Shortcut Learning. IEEE Access 10, 78726-78738 (2022). doi:10.1109/ACCESS.2022.3193700
5. Crawford Haynes, S., Johnston, P. and Elyan, E. Generalisation challenges in deep learning models for medical imagery: insights from external validation of COVID-19 classifiers. Multimedia Tools and Applications 83(31), 76753-76772 (2024). doi:10.1007/s11042-024-18543-y